

**SIMATS ENGINEERING
ASSESSMENT TOOL 3**

COURSE NAME: Design And Analysis Of Algorithms

COURSE CODE: CSA0613

COURSE OUTCOME COVERED

CO4 Solve optimization problems using dynamic programming and greedy strategies.

Levels of Rubrics

(BL-4, BL-5)

QUESTIONS: 1

Total Marks:100

**Annexure A
Comparative Analysis**

Code of Conduct:

I _P Hemanth(192472151)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P Hemanth

Q27. Knapsack Problem vs Container Loading (DP vs Greedy)

Given weights and capacity, solve using 0/1 Knapsack (DP) and Container Loading (Greedy). Compare selected items and utilization efficiency. Analyze why greedy fails for discrete optimization. Evaluate computational complexity. Discuss real-world logistics applications. Generate insights on solution strategies.

Q27. Knapsack Problem vs Container Loading (DP vs Greedy)

1. Aim

To compare the 0/1 Knapsack algorithm using Dynamic Programming with Greedy Container Loading and analyze their efficiency.

2. Objective

- Select items within a given capacity.
- Maximize value using 0/1 Knapsack.
- Maximize container weight utilization using Greedy.
- Compare the selected items and efficiency.

3. Algorithms Used

0/1 Knapsack – Dynamic Programming

Dynamic Programming evaluates different combinations of items and finds the maximum possible value without exceeding the capacity.

Container Loading – Greedy

Greedy selects items based on the smallest weight and adds them while capacity is available.

4. Sample Input

Items: 5

Weights: 10 20 30 40 50

Values: 60 100 120 140 160

Capacity: 100

5. Result

Method	Selected Items	Weight	Value	Utilization
DP Knapsack	2, 3, 4	90	360	90%
Greedy	1, 2, 3, 4	100	—	100%

6 Source Code

```
*main (2).py - C:\Users\heman\Downloads\main (2).py (3.13.15)*
File Edit Format Run Options Window Help
h = int(input("Enter number of items: "))
weights = list(map(int, input("Enter weights: ").split()))
values = list(map(int, input("Enter values: ").split()))
capacity = int(input("Enter capacity: "))
# 0/1 Knapsack - Dynamic Programming
dp = [[0] * (capacity + 1) for _ in range(n + 1)]
for i in range(1, n + 1):
    for w in range(capacity + 1):
        if weights[i-1] <= w:
            dp[i][w] = max(dp[i-1][w],
                           values[i-1] + dp[i-1][w-weights[i-1]])
        else:
            dp[i][w] = dp[i-1][w]
w = capacity
knapsack = []
for i in range(n, 0, -1):
    if dp[i][w] != dp[i-1][w]:
        knapsack.append(i)
        w -= weights[i-1]
knapsack.reverse()

# Greedy Container Loading
items = sorted(range(n), key=lambda i: weights[i])
remaining = capacity
greedy = []

for i in items:
    if weights[i] <= remaining:
        greedy.append(i + 1)
        remaining -= weights[i]
dp_weight = sum(weights[i-1] for i in knapsack)
greedy_weight = sum(weights[i-1] for i in greedy)
print("\n0/1 Knapsack (DP)")
print("Selected Items:", knapsack)
print("Total Weight:", dp_weight)
print("Total Value:", dp[n][capacity])
print("Utilization:", round(dp_weight / capacity * 100, 2), "%")
print("\nContainer Loading (Greedy)")
print("Selected Items:", greedy)
print("Total Weight:", greedy_weight)
print("Utilization:", round(greedy_weight / capacity * 100, 2), "%")
```

7 Output

```
Enter number of items: 5
Enter weights: 10 20 30 40 50
Enter values: 60 100 120 140 160
Enter capacity: 100

0/1 Knapsack (DP)
Selected Items: [1, 2, 3, 4]
Total Weight: 100
Total Value: 420
Utilization: 100.0 %

Container Loading (Greedy)
Selected Items: [1, 2, 3, 4]
Total Weight: 100
Utilization: 100.0 %

...Program finished with exit code 0
Press ENTER to exit console.
```

8. Analysis

The DP method considers different combinations and provides the maximum-value solution.

Greedy makes a locally best choice without considering all combinations.

Greedy can fail in discrete optimization because selecting an item that looks best at the current step may prevent a better combination later.

9. Complexity

Dynamic Programming:

Time: $O(n \times W)$

Space: $O(n \times W)$

Greedy:

Time: $O(n \log n)$

Space: $O(n)$

where n is the number of items and W is the container capacity.

10. Applications

- Cargo loading
- Warehouse management
- Truck and container packing
- Resource allocation
- Delivery planning

11. Conclusion

Dynamic Programming is suitable when the objective is to obtain an optimal discrete solution. Greedy is faster and simpler but may not produce the optimal combination. Therefore, DP is preferred when solution quality is more important, while Greedy is useful when speed and approximate utilization are sufficient.

RUBRICS FOR CALCULATION

Comparative Analysis							
Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Level	Marks
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario		
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues		
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts		
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided		
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand		

